

CONFIDENTIAL — RESTRICTED DISTRIBUTION



SECURITY ASSESSMENT REPORT

Esh7enly Mobile Application

PENETRATION TESTING & VULNERABILITY ASSESSMENT

APPLICATION

esh7enly_app (Flutter/Firebase)

ASSESSMENT DATE

September 2026

FIREBASE PROJECT ID

esh7enly-cf37d

DOCUMENT VERSION

v1.0 — Final

TESTING METHODOLOGY

Black-box + White-box (Source)

REPORT CLASSIFICATION

CONFIDENTIAL

⚠ CONFIDENTIAL — DO NOT DISTRIBUTE

This document contains sensitive security findings.

Unauthorised disclosure may compromise the security of the assessed system.

Table of Contents

PREAMBLE

— Executive Summary	3
— Scope & Engagement Details	3
— Risk Rating Methodology	4
— Findings Overview	4

TEST RESULTS

01 Application Architecture & Route Analysis	5
02 Network & Port Scan	6
03 HTTP Response & Security Headers	7
04 Directory Enumeration	8
05 Authentication Testing	9
06 API Security & Key Exposure	11
07 IDOR — Insecure Direct Object Reference	12
08 Password Reset Vulnerability Testing	13
09 File Upload Security	14
10 Remote Code Execution (RCE) & Injection	15

CONCLUSIONS

11 Vulnerability Risk Matrix	16
12 Remediation Roadmap	17
13 Tools & Methodology	18

Scope: Port Scanning · HTTP Response Analysis · Directory Enumeration · Authentication Testing · API Security · IDOR · Password Reset · File Upload · Remote Code Execution · Static Code Review

Executive Summary

Penetration Testing & Vulnerability Assessment — Esh7enly Flutter/Firebase Application

Assessment Overview

A comprehensive black-box and white-box penetration test was conducted against the **Esh7enly** mobile application — an EV charging session management platform built with Flutter (Dart) and Firebase. The assessment spanned 10 attack categories including network reconnaissance, authentication bypass, API abuse, IDOR, password reset attacks, file upload vulnerabilities, and remote code execution testing.

The assessment identified **1 Critical**, **4 High**, **7 Medium**, and **3 Low** severity findings across the application's codebase and backend infrastructure. The most severe vulnerability — **missing authentication guards on all protected routes** — allows an unauthenticated attacker to directly access payment, wallet, and session management features without logging in.

1

CRITICAL

4

HIGH

7

MEDIUM

3

LOW

— Scope & Engagement Details

PARAMETER	DETAILS
Application Name	esh7enly_app
Platform	Flutter (Dart SDK ^3.10.8) — Android / iOS / Web

PARAMETER	DETAILS
Backend	Firebase Authentication + Firestore + Firebase Storage
Firebase Project	esh7enly-cf37d
Hosting URL	https://esh7enly-cf37d.firebaseio.com
Assessment Type	Black-box (API) + White-box (Dart source code review)
Assessment Date	September 2026
Testing Methodology	OWASP Mobile Top 10, OWASP Web Top 10, Firebase Security Best Practices
In Scope	Firebase Auth REST API, Firestore REST API, Firebase Storage, Flutter source code, Firebase Hosting
Out of Scope	Google infrastructure internals, third-party services, physical access attacks

Risk Rating Methodology

All findings are rated using an approximated CVSS v3.1 score combined with contextual business impact analysis.

RATING	CVSS RANGE	DESCRIPTION	SLA
CRITICAL	9.0 – 10.0	Direct exploitation with immediate, significant impact on confidentiality, integrity, or availability	Immediate
HIGH	7.0 – 8.9	Exploitation possible with moderate conditions; significant data exposure or system compromise	Before Launch
MEDIUM	4.0 – 6.9	Exploitation requires specific conditions; moderate impact on security posture	Next Sprint
LOW	0.1 – 3.9	Limited exploitability; minor security improvement recommended	Best Effort

01 Application Architecture & Route Analysis

Static code analysis of the Flutter application's source tree to understand the attack surface, routing behaviour, and authentication enforcement points.

1.1 Technology Stack

Component	Technology	Version
Framework	Flutter (Dart)	SDK ^3.10.8
Backend-as-a-Service	Firebase	Core ^4.14.0
Authentication	Firebase Authentication (Email/Password)	^6.1.4
State Management	flutter_bloc (Cubit pattern)	^9.1.1
Navigation/Routing	go_router	^15.1.2
Social Auth	UI present — NOT implemented (TODO)	—

1.2 Route Security Analysis

🚨 CRITICAL FINDING — V-01

The GoRouter configuration contains **zero authentication redirect guards** on any protected route. An attacker can deep-link or URL-manipulate directly to `/home` , `/payment` , `/wallet` , and `/profile` **without being authenticated**.

Route Path	Route Name	Auth Guard	Risk
<code>/</code>	splash	None required	—
<code>/login</code>	onboarding	Public	—
<code>/register</code>	register	Public	—
<code>/home</code>	home	NO GUARD	High

ROUTE PATH	ROUTE NAME	AUTH GUARD	RISK
/select-car	selectCar	NO GUARD	High
/payment	payment	NO GUARD	Critical
/charging-complete	chargingComplete	NO GUARD	High
/sessions	sessions	NO GUARD	High
/wallet	wallet	NO GUARD	Critical
/profile	profile	NO GUARD	High
/location	location	NO GUARD	Medium

1.3 Vulnerable Code — Router Configuration

```
// lib/core/router/app_router.dart (CURRENT - VULNERABLE)
GoRoute(
  path: RoutesName.home,      // '/home'
  name: 'home',
  builder: (context, state) => const HomeView(),
  // ⚠️ No redirect guard – unauthenticated access is possible
),
GoRoute(
  path: RoutesName.payment,   // '/payment'
  name: 'payment',
  builder: (context, state) => const PaymentView(),
  // ⚠️ No redirect guard – financial route fully unprotected!
),
```

1.4 Other Code-Level Issues Discovered

- **debugLogDiagnostics: true** — Router emits verbose debug logs in production builds
- **device_preview: ^1.3.1** in production `dependencies` (should be `dev_dependencies`)
- **Guest Login dead code** — `onTap: () { //TODO: handle log in as a guest }` UI button exposed with no backend
- **No session invalidation** — No `FirebaseAuth.instance.authStateChanges()` stream listener or logout token revocation observed

02 Network & Port Scan

Tool: Nmap 7.98 (-sV -sC) | **Targets:** firestore.googleapis.com, identitytoolkit.googleapis.com, securetoken.googleapis.com, firebase.googleapis.com

2.1 Port Scan Results

PORT	STATE	SERVICE	NOTES	RISK
80/tcp	OPEN	HTTP (gws)	Immediately redirects to HTTPS — acceptable	PASS
443/tcp	OPEN	HTTPS (gws)	TLS, HTTP/2, gRPC supported	PASS
3000/tcp	FILTERED	—	Not accessible from external network	PASS
4000/tcp	FILTERED	—	Not accessible	PASS
5000/tcp	FILTERED	—	Not accessible	PASS
8080/tcp	FILTERED	—	Not accessible	PASS
8443/tcp	FILTERED	—	Not accessible	PASS
9000/tcp	FILTERED	—	Firebase Emulator port — not internet-exposed	PASS

2.2 TLS / Certificate Analysis

PROPERTY	VALUE
Certificate Authority	edgectrl.googleapis.com
Validity	Valid until 2026-11-02
Protocols Supported	TLS 1.2, TLS 1.3, HTTP/2, gRPC-exp

PROPERTY	VALUE
Protocol Downgrade	Not observed — TLS enforced

FINDING

Network attack surface is minimal. Only standard ports 80 and 443 are externally exposed. Port 80 enforces HTTPS redirect. No custom backend infrastructure discovered. Google's hardened network edge handles all traffic.

03 HTTP Response & Security Headers

Target: https://esh7enly-cf37d.firebaseio.com | **Method:** PowerShell HttpWebRequest + manual header audit

3.1 HTTP Response

HTTP/1.1 404 Not Found

Note: No web app deployed to Firebase Hosting yet – 404 is expected.
The Flutter app is currently mobile-only.

3.2 Security Headers Audit

HEADER	PRESENT?	RISK IF MISSING
Strict-Transport-Security	PRESENT ✓	MITM downgrade attacks
Server	HIDDEN ✓	Server fingerprinting
X-Powered-By	HIDDEN ✓	Technology disclosure
Content-Security-Policy	MISSING ⚠	Cross-Site Scripting (XSS)
X-Content-Type-Options	MISSING ⚠	MIME-type sniffing attacks
X-Frame-Options	MISSING ⚠	Clickjacking attacks
X-XSS-Protection	MISSING ⚠	Legacy browser XSS bypass
Referrer-Policy	MISSING ⚠	URL leakage in Referer header

HEADER	PRESENT?	RISK IF MISSING
Permissions-Policy	MISSING ⚠️	Browser feature abuse (camera, mic, geo)

⚠️ MEDIUM — V-08

Six security headers are absent. Once the web app is deployed to Firebase Hosting, these headers MUST be added via `firebase.json` to prevent XSS, clickjacking, and MIME-sniffing attacks.

04 Directory Enumeration

Tool: Custom PowerShell wordlist scanner | **Paths tested:** 34 common admin/config paths

4.1 Results Summary

Total paths scanned: 34
Results: ALL 34 → 404 Not Found
No exposed directories, admin panels, or configuration files found.

PATH	STATUS
/admin , /wp-admin , /dashboard	404 — NOT EXPOSED ✓
/.env , /.git , /.gitignore	404 — NOT EXPOSED ✓
/firebase.json , /google-services.json	404 — NOT EXPOSED ✓
/api/v1 , /api/users , /graphql	404 — NOT EXPOSED ✓
/home , /payment , /wallet	404 — NOT EXPOSED ✓

i NOTE

No directory enumeration vulnerabilities found on the hosting layer. However, `google-services.json` and `firebase_options.dart` exist **inside the compiled APK/IPA bundle**. An attacker who extracts the app binary via APK decompilation (e.g., `apktool` , `jadx`) can retrieve all hardcoded API keys and Firebase project configuration.

05 Authentication Testing

Target: Firebase Identity Toolkit REST API v1 | **Tests:** 8 manual test cases covering login, registration, brute-force, token validation, and anonymous auth

5.1 Test Results

#	TEST	EXPECTED	OBSERVED	RESULT
T1	Login with wrong credentials	Generic error (no user enumeration)	INVALID_LOGIN_CREDENTIALS — generic ✓	PASS
T2	Brute-force (5 rapid attempts)	Rate limiting / lockout triggered	All 5 attempts return same error — no lockout triggered	MEDIUM
T3	Unrestricted account creation	CAPTCHA or App Check required	Arbitrary email → 200 OK with new localId	HIGH ⚠
T4	Password reset enumeration	Always return 200 (no email disclosure)	200 for both existing and non-existent emails ✓	PASS
T5	Firestore unauthenticated access	Blocked without auth token	404 — DB disabled / rules block access ✓	PASS
T6	Firebase Storage unauthenticated access	Blocked without auth token	404 — Storage not configured ✓	PASS
T7	Fake/forged JWT token lookup	Token rejected	INVALID_ID_TOKEN ✓	PASS
T8	Anonymous sign-in	Disabled — should be blocked	ADMIN_ONLY_OPERATION — disabled ✓	PASS

5.2 Password Policy — Code Review Finding

```
// lib/core/utils/app_validators.dart
static String? password(String? value) {
  if (value!.length < 6) return 'Password must be at least 6 characters';
  // ⚠ No complexity requirements: 'aaaaaa' is a valid password!
}
```

● HIGH — V-03

Minimum password length of only 6 characters with no complexity requirements (no uppercase, no digit, no special character enforcement). Passwords such as `aaaaaa` ,

123456 , or passw0 pass validation.

06 API Security & Key Exposure

6.1 Firebase API Endpoint Analysis

ENDPOINT	METHOD	AUTH	FINDING
<code>/v1/accounts:signInWithPassword</code>	POST	API Key	Open — brute-forceable without App Check
<code>/v1/accounts:signUp</code>	POST	API Key	Open — allows unlimited fake account creation
<code>/v1/accounts:sendOobCode</code>	POST	API Key	Open — safe (no email enumeration)
<code>/v1/accounts:lookup</code>	POST	ID Token	Validates token correctly ✅
<code>/v1/accounts:update</code>	POST	ID Token	Requires valid token ✅

6.2 Hardcoded API Keys — Critical Exposure

● HIGH — V-04

All Firebase API keys are hardcoded in `firebase_options.dart` and compiled into the app binary. APK decompilation (using jadx, apktool) trivially recovers them. Without API Key restrictions in Google Cloud Console and Firebase App Check enforcement, these keys can be abused to:

- Create unlimited fake accounts (quota exhaustion)
- Send bulk password reset spam emails to all users
- Enumerate registered users via signup endpoint
- Brute-force login at scale

PLATFORM

API KEY (REDACTED)

Web / Windows

`AIzaSyBcivAjtqnWP3d-Zhue***`

PLATFORM	API KEY (REDACTED)
Android	AIzaSyCpJ1R2rQ8SmWN3iShx***
iOS	AIzaSyBNf2s7j9eTCygZoGA2***

Note: Firebase API keys are by design public identifiers, not secrets. The actual risk lies in unrestricted API key usage and missing App Check enforcement.

07 IDOR — Insecure Direct Object Reference

Tests probe whether an attacker can access or modify another user's data by guessing or substituting object identifiers (user IDs, document paths, session IDs).

7.1 IDOR Test Results

TEST	REQUEST	RESPONSE	FINDING
Admin user lookup by localId	POST /v1/projects/.../accounts:lookup	403 PERMISSION_DENIED	BLOCKED ✓
Firestore doc access by guessed UID	GET /databases/(default)/documents/users/{uid}	403 SERVICE_DISABLED	BLOCKED ✓
Collection enumeration (9 collections)	GET on users, sessions, payments, wallets, etc.	All → 403	BLOCKED ✓

⚠ HIGH — V-05 (LATENT RISK)

Firestore is currently disabled — all IDOR tests are blocked by the service being off. However, once Firestore is activated for production use (user sessions, payments, wallets), IDOR becomes a **Critical threat** if Security Rules are written incorrectly. A common mistake is:

```
// DANGEROUS — allows ANY authenticated user to read ALL users!
match /users/{userId} {
  allow read, write: if request.auth ≠ null;
}
```

The correct rule must scope access to the document owner:

```
match /users/{userId} {  
  allow read, write: if request.auth.vid == userId;  
}
```

08 Password Reset Vulnerability Testing

8.1 Test Results

TEST	ATTACK TYPE	RESULT	FINDING
Reset flooding (same email × 3)	Email bombing / inbox DoS	3 × 200 OK — no throttle	VULNERABLE ⚠️
Reset spray (5 different accounts)	Bulk reset spam — no per-IP limit	5 × 200 OK — no rate limit	VULNERABLE ⚠️
Host header injection	Reset URL poisoning via X-Forwarded-Host	Firebase pins reset domain — not vulnerable	NOT VULNERABLE ✅
Fake OOB code bypass	Password change without valid reset token	400 INVALID_OOB_CODE	BLOCKED ✅

⚠️ MEDIUM — V-06, V-07

No rate limiting was enforced on the password reset endpoint for either single-email flooding or multi-target spray. An attacker can flood any user's inbox with reset emails, causing inbox DoS and potential account takeover confusion. Firebase did not throttle 3 consecutive same-email requests or 5 rapid cross-account requests.

09 File Upload Security

Target: Firebase Storage (firebasestorage.googleapis.com) | **Tests:** 4 test cases, 7 malicious filenames

9.1 Upload Test Results

TEST	PAYLOAD	RESPONSE	FINDING
Unauthenticated upload	Plain text file, no auth token	404	BLOCKED ✓
Path traversal filenames	../ ../ ../etc/passwd , URL-encoded variants	All → 404	BLOCKED ✓
Dangerous extensions	shell.php , malware.exe , test.html	All → 404	BLOCKED ✓
SVG/XSS MIME bypass	SVG with embedded <script> tag	404	BLOCKED ✓
Public bucket listing	GET /o (no auth)	404	BLOCKED ✓

i LATENT — V-13

File upload is currently safe because Firebase Storage is not yet configured. Once Storage Rules are set and the app enables uploads (e.g., profile photos), these risks MUST be addressed: file type allowlisting, SVG XSS isolation, size limits (<5 MB), and malware scanning via Cloud Functions.

10 Remote Code Execution & Injection

10.1 RCE Test Results

TEST	VECTORS	RESULT
Cloud Functions discovery	78 URLs (6 regions × 13 function names)	ALL 404 — NO FUNCTIONS DEPLOYED ✓
App Engine / Cloud Run	appspot.com, Cloud Run services API	404 / 401 — NO SERVICES DEPLOYED ✓
NoSQL Injection (6 payloads)	{"\$gt":""} , OR 1=1 , SQL/NoSQL operators	ALL REJECTED — EMAIL VALIDATION BLOCKS INJECTION ✓

TEST	VECTORS	RESULT
Server-Side Template Injection (7 payloads)	<code>{{7*7}}</code> , <code>\${7*7}</code> , Jinja2 <code>__import__</code>	STORED AS LITERAL STRINGS — NOT EVALUATED ✓
Stored XSS via display name	<code></code>	STORED — LATENT RISK IF RENDERED IN HTML ⚠

⚠ MEDIUM — V-10 (LATENT STORED XSS)

Firebase Auth accepts and stores raw HTML characters (including `<script>` , `onerror=` , `"` , `'`) in the `displayName` field without sanitization. If any future web admin dashboard or email template renders this field as raw HTML, a stored XSS payload will execute. Sanitize display names on both Dart-side input and any HTML output layer.

11 Vulnerability Risk Matrix

ID	VULNERABILITY	CATEGORY	SEVERITY	CVSS	STATUS
V-01	No route-level authentication guards on GoRouter — all protected routes accessible without login	Auth / Access Control	CRITICAL	9.1	OPEN
V-02	Unrestricted account creation via public API key — no App Check or CAPTCHA	Auth / API	HIGH	7.5	OPEN
V-03	Weak password policy — minimum 6 chars, no complexity requirements	Authentication	HIGH	7.3	OPEN
V-04	API keys not restricted in Google Cloud Console — exposed in compiled APK	Configuration	HIGH	7.0	UNVERIFIED
V-05	Latent IDOR — Firestore Security Rules not written; any authenticated user could access all users' data	IDOR	HIGH	7.0	LATENT
V-06	Password reset email flooding — no rate limit on same-email repeat requests	Password Reset	MEDIUM	6.5	OPEN

ID	VULNERABILITY	CATEGORY	SEVERITY	CVSS	STATUS
V-07	Password reset spray — no per-IP throttle on cross-account reset requests	Password Reset	MEDIUM	6.2	OPEN
V-08	Missing security headers — CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy absent	Configuration	MEDIUM	6.1	OPEN
V-09	No brute-force lockout on short-burst login attempts	Authentication	MEDIUM	5.9	OPEN
V-10	Stored XSS latent risk — display name accepts raw HTML characters	XSS / Injection	MEDIUM	5.4	LATENT
V-11	Social login buttons exposed in UI — unimplemented backend (dead code)	Authentication	MEDIUM	4.3	OPEN
V-12	No logout / token invalidation logic — no authStateChanges() stream listener	Session Management	MEDIUM	4.3	OPEN
V-13	File upload — no Firebase Storage Rules defined; will be misconfigured when bucket is activated	File Upload	MEDIUM	4.3	LATENT
V-14	<code>debugLogDiagnostics: true</code> in production GoRouter configuration	Configuration	LOW	3.7	OPEN

ID	VULNERABILITY	CATEGORY	SEVERITY	CVSS	STATUS
V-15	<code>device_preview</code> package in production <code>dependencies</code> (should be <code>dev_dependencies</code>)	Configuration	LOW	3.1	OPEN

12 Remediation Roadmap

**CRITICAL
IMMEDIATE**

V-01 — Add GoRouter Authentication Guards

Add a `redirect` callback to every protected route. Check `FirebaseAuth.instance.currentUser` and redirect unauthenticated users to the login screen.

```
// app_router.dart – Add this redirect to all protected GoRoutes
redirect: (context, state) {
  final isLoggedIn = FirebaseAuth.instance.currentUser != null;
  final isPublic = [RoutesName.login, RoutesName.register, RoutesName.splash]
    .contains(state.matchedLocation);
  if (!isLoggedIn && !isPublic) return RoutesName.login;
  return null;
},
```

**HIGH
PRE-LAUNCH**

V-02, V-04 — Enable Firebase App Check & Restrict API Keys

- Enable **Firebase App Check** with reCAPTCHA (web) and Play Integrity / DeviceCheck (mobile) to block automated API abuse
- Restrict API keys in **Google Cloud Console** → **Credentials** — add Android SHA, iOS Bundle ID, and HTTP referrer restrictions
- Consider using environment-based secrets instead of hardcoding in source

**HIGH
PRE-LAUNCH**

V-03 — Strengthen Password Policy

```
// app_validators.dart – Replace password validator
static String? password(String? value) {
  if (value == null || value.length < 8)
    return 'Password must be at least 8 characters';
  if (!value.contains(RegExp(r'[A-Z]')))
    return 'Must contain at least one uppercase letter';
  if (!value.contains(RegExp(r'[0-9]')))
    return 'Must contain at least one number';
  if (!value.contains(RegExp(r'[@#\$%^&*]')))
    return 'Must contain at least one special character';
}
```

```
return null;
}
```

HIGH
PRE-FIRESTORE

V-05 — Write User-Scoped Firestore Security Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId}/{document=**} {
      allow read, write: if request.auth.uid == userId;
    }
    match /sessions/{sessionId} {
      allow read, write: if request.auth.uid == resource.data.userId;
    }
    match /wallets/{walletId} {
      allow read, write: if request.auth.uid == resource.data.ownerId;
    }
  }
}
```

MEDIUM
NEXT SPRING

V-06, V-07 — Password Reset Rate Limiting

- Add a 60-second client-side cooldown before the user can request another reset email
- In Firebase Console → Authentication → Settings → Quota: enable rate limits
- Add reCAPTCHA v3 before the reset form submission

MEDIUM
NEXT SPRING

V-08 — Add Security Headers via firebase.json

```
"hosting": {
  "headers": [{
    "source": "**",
    "headers": [
      {"key": "Content-Security-Policy", "value": "default-src 'self'"},
      {"key": "X-Content-Type-Options", "value": "nosniff"},
      {"key": "X-Frame-Options", "value": "SAMEORIGIN"},
      {"key": "Referrer-Policy", "value": "strict-origin-when-cross-origin"},
      {"key": "Permissions-Policy", "value": "camera=(), microphone=()"}
    ]
  }]
}
```

MEDIUM

V-12 — Implement Session Management

NEXT SPRINT

- Listen to `FirebaseAuth.instance.authStateChanges()` stream and redirect to login on sign-out
- Call `FirebaseAuth.instance.signOut()` on logout with token invalidation
- Add session timeout — force re-authentication after inactivity period

**LOW
BEST EFFORT****V-14, V-15 — Build Config Cleanup**

- Move `device_preview` from `dependencies` to `dev_dependencies`
- Disable `debugLogDiagnostics` using build-mode checks: `kDebugMode ? true : false`
- Add email verification requirement before granting access to protected routes

13 Tools & Methodology

PHASE	TOOL / METHOD	TESTS CONDUCTED
Port Scan	Nmap 7.98 (<code>-sV -sC</code>)	4 Firebase hosts × 8 ports = 32 port checks
HTTP Analysis	PowerShell <code>HttpWebRequest</code> + manual header audit	9 security headers checked
Directory Enumeration	Custom PowerShell wordlist scanner	34 common admin/config/sensitive paths
Authentication Testing	Firebase Identity Toolkit REST API (manual)	8 test cases
API Exploration	Firebase Auth, Firestore, Storage REST APIs	5 endpoints analysed
IDOR Testing	Firestore REST API + localId enumeration	3 test vectors, 9 Firestore collections
Password Reset	Firebase OOB Code API	4 test cases (flooding, spray, host injection, fake token)
File Upload	Firebase Storage REST API	4 test cases, 7 malicious filenames/paths
RCE / Injection	Cloud Functions URL scan + manual injection	78 endpoint URLs, 6 NoSQL payloads, 7 SSTI payloads
Static Code Review	Manual Dart/Flutter source analysis	All files under <code>lib/</code> reviewed

Testing Standards Applied

- **OWASP Mobile Security Testing Guide (MSTG)** — Mobile-specific vulnerability categories
- **OWASP Top 10 Web** — Injection, Broken Auth, Security Misconfiguration, Insecure Design

- **OWASP API Security Top 10** — API key exposure, rate limiting, IDOR
- **Firebase Security Best Practices** — Security Rules, App Check, API key restrictions
- **CVSS v3.1** — Vulnerability scoring methodology

Limitations & Assumptions

ASSESSMENT LIMITATIONS

- Firestore database was not enabled during testing — Firestore IDOR rules cannot be fully verified until the service is activated
- Firebase Storage bucket was not configured — upload rules and MIME validation untestable until Storage is set up
- No Cloud Functions are deployed — server-side code injection surface is zero at this stage
- Testing was conducted in a pre-production environment — production may differ
- Physical device security (rooting, SSL pinning bypass) was not in scope

Conclusion

The Esh7enly application has a **solid Firebase backend configuration** — Firestore, Storage, and anonymous auth are all appropriately restricted at the service level. The network attack surface is minimal and TLS is properly enforced.

However, the application has **critical gaps at the application layer**: the complete absence of route-level authentication guards is the most severe finding and must be remediated before any user data or financial information is handled. Additionally, weak password policy, missing App Check, and the latent IDOR risk in Firestore rules all require immediate attention before the production launch.

Addressing the 5 Critical/High findings before launch will significantly improve the application's security posture and protect user financial and session data.

Document Control

Report ID: PENTEST-ESH7ENLY-2026 | Version: 1.0 Final | Date: September 2026

Classification: **CONFIDENTIAL — RESTRICTED**

This report is intended solely for the Esh7enly development team. Unauthorised disclosure is prohibited.

All tests were conducted with authorisation in a controlled assessment environment.

